DS551 Fall 25 - Epidemic Engine

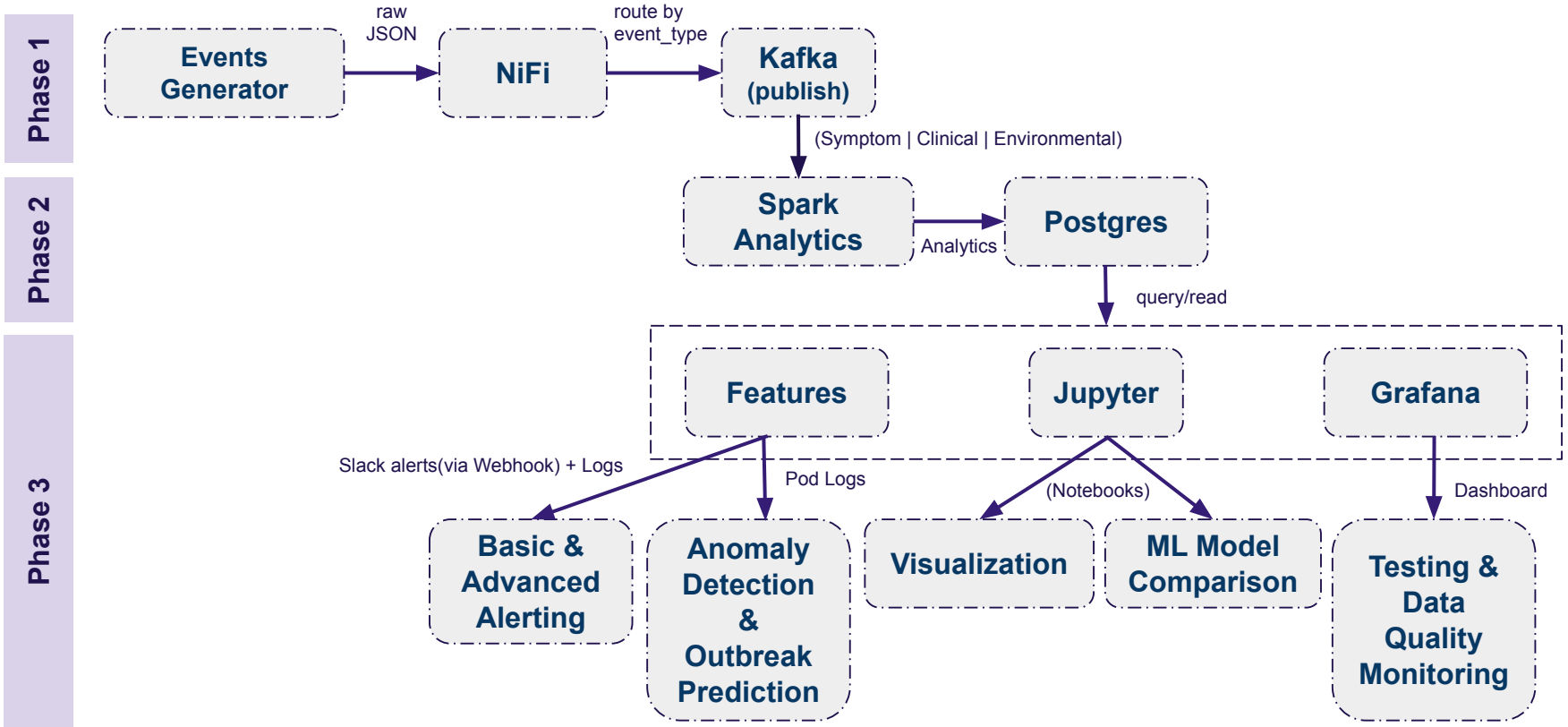
(Team 07)

Team Members:

- **Joshua Lee**
- **Pranshu Swaroop**
- **Samritha Aadhi Ravikumar**
- **Shravani Maskar**

Date: 12/16/2025

System Architecture



Features Implemented

Category	Feature	Status	Team Member(s)	Points
Phase 1	**Data Ingestion**	✅ Completed	**Joshua**	60 pts
Phase 2	**Analytics & Storage**	✅ Completed	**Shravani**	40 pts
Phase 3	**Integration & Features**			
Category A	**Basic Alerting**	✅ Working	**Samritha**	20 pts
	Advanced Alerting	✅ Working		30 pts
Category B	**Anomaly Detection**	✅ Working	**Samritha**	25 pts
	Outbreak Prediction	✅ Working		35 pts
Category C	**Visualisation**	✅ Completed	**Samritha**	15 pts
Category D	**Comprehensive Testing Suite**	✅ Completed	**Pranshu**	20 pts
	Data Quality Monitoring	✅ Completed		25 pts
Extra Credit	**Grafana Dashboard**	✅ Completed	**Pranshu**	15–30 pts
	ML Model Comparison	✅ Completed	**Pranshu & Joshua**	15–35 pts

Technical Deep-Dive

Micro-batch Data Quality Monitoring (Spark foreachbatch → Postgres)

```
def dq_batch(df, epoch_id):
    print(f"[DQ] Running Data Quality for batch {epoch_id}")
    dq_metrics_df = run_dq_for_df(df)
    dq_metrics_df.show(truncate=False)
    write_metrics_to_db(dq_metrics_df)


dq_query = severity_df \
    .writeStream \
    .outputMode("append") \
    .foreachBatch(dq_batch) \
    .trigger(processingTime="30 seconds") \
    .start()
```

Isolated batch-level validation function

Applies validation rules & aggregates failures

Persists data quality metrics to Postgres

Runs data quality validation once per micro-batch

Continuous monitoring with fixed reporting interval

Lessons Learned & Gen AI

- **Challenges:** Debugging across pods needed oc logs + “topic → DB row” checks.
- **If starting over:** Freeze schema + topic contracts early (event_type + required fields) to prevent downstream breaks.
- **Given more time:** Add reliability and observability (validation, error handling, lag + DB write metrics).
- **Recommendation:** Add Dead-letter queue(DLQ) + replay + trace_id for debuggable, resilient streaming.

- **Used for learning/debugging:** Clarified Spark streaming concepts and helped interpret error logs; we validated by rerunning in OpenShift.
- **Most helpful:** Converting log messages into a checklist (config, schema, permissions, resources).
- **Limits:** Couldn’t know our cluster/YAML/quota setup—final fixes came from oc describe, pod logs, and config edits.

Thank You!

Q&A